

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – SAST SIMULATION

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Static Application Security Testing (SAST) using synthetic data		
Student Name:	Adhithyan S	Reg. No.:	192421102
Section / Batch:		Date:	

Instructions to Students

- Ensure all diagrams and workflow illustrations are original, clear, and professionally formatted.
- Include all editable source files (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit both the GitHub repository link and the final PDF report through Google Classroom before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

SAST SIMULATION	Creative Things and concept grasping	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

SAST Simulation Task

Static Application Security Testing — Payment Gateway Application

Training / Simulation Exercise — Confidential

1. Objective

This exercise simulates a Static Application Security Testing (SAST) engagement against a fictional online payment gateway codebase. Participants act as an application security analyst: reviewing source code, interpreting simulated SAST tool output, prioritizing findings by risk, and proposing remediations that align with secure coding and PCI DSS practices.

2. Scope

- Target: a simulated payment gateway comprising an API gateway, authentication service, payment processing service, transaction ledger, and transaction database.
- In scope: server-side application source code for the services above (illustrative snippets provided in Section 5).
- Out of scope: infrastructure penetration testing, third-party PSP internals, physical security.
- Tooling (simulate with any SAST engine of your choice, e.g. Semgrep, SonarQube, Checkmarx, or a classroom sandbox): static analysis only — no live traffic is generated against a real system.

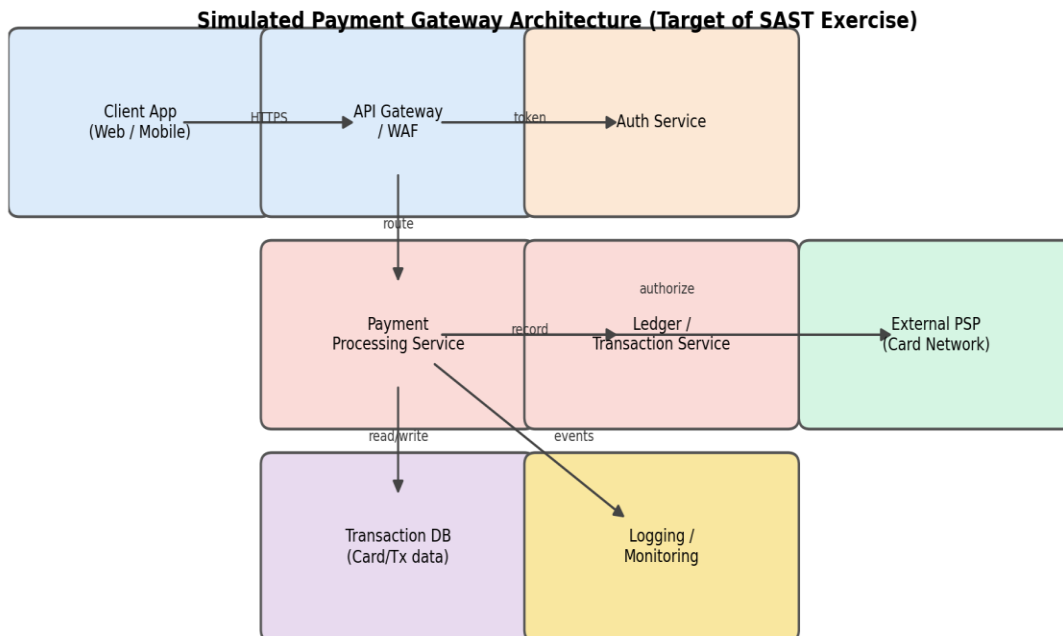


Figure 1 — Simulated payment gateway architecture used as the SAST target.

3. Task Instructions

- Review the architecture in Figure 1 and identify the components most likely to process sensitive cardholder data.
- Examine each code snippet in Section 5 and classify the vulnerability using CWE and OWASP Top 10 categories.
- Complete the findings table in Section 6 (or reproduce it) with a CVSS-style severity rating and a one-line remediation for each item.
- Map each finding to the relevant PCI DSS requirement it violates (see Section 7).

- Produce a short remediation plan prioritizing fixes by severity and exploitability.
- Re-run (simulate) the SAST scan after applying fixes and confirm the finding is closed.

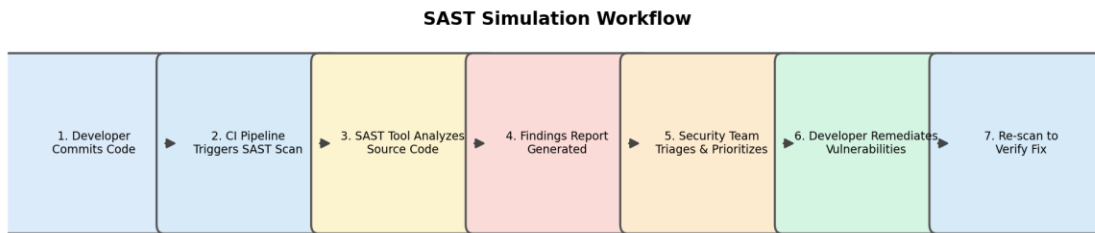


Figure 2 — SAST simulation workflow to follow during this exercise.

4. Simulated Codebase Under Test

The snippets below are illustrative examples written for this training exercise. They represent common vulnerability patterns found in real payment applications and are intentionally simplified for analysis practice.

4.1 Payment Service — Transaction Lookup (SQL Injection)

```
// payment-service/src/transactions.js
app.get('/api/transactions', (req, res) => {
  const orderId = req.query.orderId;
  const query = "SELECT * FROM transactions WHERE order_id = '" + orderId + "'";
  db.execute(query, (err, rows) => {
    if (err) return res.status(500).send(err.message);
    res.json(rows);
  });
});
```

Finding: user input is concatenated directly into a SQL statement without parameterization, allowing SQL injection via the `orderId` parameter.

4.2 Config — Hardcoded Credentials

```
// payment-service/src/config.js
module.exports = {
  pspApiKey: 'sk_live_49aF3kLp9zQ2mN8vTgH1',
  pspApiSecret: 'ps_secret_7hT2kLm9NpQr4vXz',
  dbPassword: 'P@ssw0rd123',
};
```

Finding: live-looking API keys and database credentials are hardcoded in source, so anyone with repository access (or a leaked build artifact) obtains production secrets.

4.3 Logging — Sensitive Data Exposure

```
// payment-service/src/processPayment.js
```

```
function processPayment(cardNumber, expiry, cvv, amount) {  
  logger.info(`Processing payment for card ${cardNumber}, amount ${amount}`);  
  return psp.charge({ cardNumber, expiry, cvv, amount });  
}
```

Finding: full primary account numbers (PAN) are written to application logs in plaintext, violating PCI DSS Requirement 3 (protect stored cardholder data).

4.4 Auth Service — Weak Password Hashing

```
// account-service/src/auth.js  
const crypto = require('crypto');  
function hashPassword(password) {  
  return crypto.createHash('md5').update(password).digest('hex');  
}
```

Finding: MD5 is a fast, collision-prone hash unsuitable for password storage; no per-user salt is applied, enabling rainbow-table and brute-force attacks.

4.5 Input Validation — Card Number Format

```
// payment-service/src/validate.js  
function isValidCardNumber(input) {  
  return /^[12,19]$/.test(input); // accepts any characters, not digits only  
}
```

Finding: the validation regex checks length only and accepts non-numeric characters, pushing format validation downstream to the PSP instead of failing fast.

4.6 Service-to-Service Client — TLS Validation Disabled

```
// ledger-service/src/httpClient.js  
const https = require('https');  
const agent = new https.Agent({ rejectUnauthorized: false });  
module.exports = { agent };
```

Finding: disabling certificate validation on internal HTTPS calls exposes the ledger service to man-in-the-middle attacks even on 'trusted' internal networks.

5. Simulated SAST Findings Report

Consolidated output an analyst would expect a SAST tool to produce for the snippets in Section 4. Use this as a reference answer key or as a starting template if generating your own report.

ID	Severity	Description	CWE
F-01	Critical	Hardcoded PSP API key and secret embedded directly in payment-service source code.	CWE-798
F-02	Critical	SQL query in transaction lookup built via string concatenation of user-supplied order ID (SQL Injection).	CWE-89
F-03	High	Card numbers written to application logs in plaintext during payment processing.	CWE-532
F-04	High	Passwords hashed with MD5 and no salt in the account-service authentication module.	CWE-916
F-05	Medium	Card number format validated with a weak regex that accepts non-numeric input, relying on downstream PSP validation.	CWE-20
F-06	Medium	Verbose error messages returned to the client expose internal stack traces on failed transactions.	CWE-209
F-07	Low	TLS certificate validation disabled in the internal HTTP client used for service-to-service calls.	CWE-295

6. PCI DSS Mapping

- F-01, F-02 → Requirement 6: develop and maintain secure systems and applications (secure coding, secrets management).
- F-03 → Requirement 3: protect stored cardholder data (no PAN in logs).
- F-04 → Requirement 8: strong authentication and credential storage.
- F-05, F-06 → Requirement 6.5: address common coding vulnerabilities (input validation, error handling).
- F-07 → Requirement 4: encrypt transmission of cardholder data across networks, including internal segments.

7. Remediation Guidance

- Replace string-concatenated SQL with parameterized queries / prepared statements or an ORM.
- Move all secrets to a managed vault (e.g., environment-injected secrets manager) and rotate any keys that were ever committed to source control.
- Mask or omit PAN, CVV, and full track data from all log output; log only a tokenized or truncated reference.
- Replace MD5 with a modern password hashing algorithm (bcrypt, scrypt, or Argon2) with a per-user salt and appropriate work factor.
- Validate card numbers with a strict numeric-and-length check plus a Luhn checksum before submission to the PSP.
- Return generic error messages to clients; log full stack traces only server-side.
- Re-enable TLS certificate validation for all internal and external HTTPS clients.

8. Deliverables

- Completed findings table (Section 5) with severity justification for each item.
- PCI DSS mapping for each finding.
- Screenshot or export of the simulated 'clean' re-scan after fixes are applied.

Code of Conduct

I Adhithyan S (192421102) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand